



SELECTED WORK

Four products, built **end to end**.

Each of these went from an empty repository to something people use: the product thinking, the interface, the API, the database, the infrastructure and the releases. Three pages each — what it does, how it works, and the decisions I would defend.

01

Aplyfy

An AI job-application platform. It finds matching jobs, rewrites the CV for each one, writes the letter and email, and can fill in the employer's form on your behalf.

pages 2-4

[Open →](#)

React 19

NestJS

Postgres

AWS Lambda

Bedrock · Gemini

02

Visorun

A CV anonymisation platform for recruiters. It strips every identifying detail from a candidate's CV and returns a branded PDF — wrapped in plans, quotas and Stripe billing.

pages 5-7

[Open →](#)

React 19

NestJS

Firestore

Stripe

Gemini

03

Stuzanne

A workshop system for a jewellery maker. Every commissioned piece is tracked from quote to collection, and every gram of gold issued is reconciled against what comes back.

pages 8-10

[Open →](#)

.NET 6

MySQL

React 18

Electron

Docker

04

LR Car Services

The driver's app for a dispatcher-run car service. Multi-stop routes, waiting time billed apart from driving time, and a location feed that survives the app being killed.

pages 11-13

[Open →](#)

React Native

Firebase

Cloud Functions

Background GPS



SELECTED WORK · 01

Aplyfy

An **AI job-application platform** — designed, built and run by me.

One profile in — matched job, rewritten CV, cover letter, ready-to-send email, finished PDFs out. Under a minute, English or German. Autopilot files it for you.

My role Everything: product and UI, API, AI pipeline, database, AWS infra, CI and deploys.

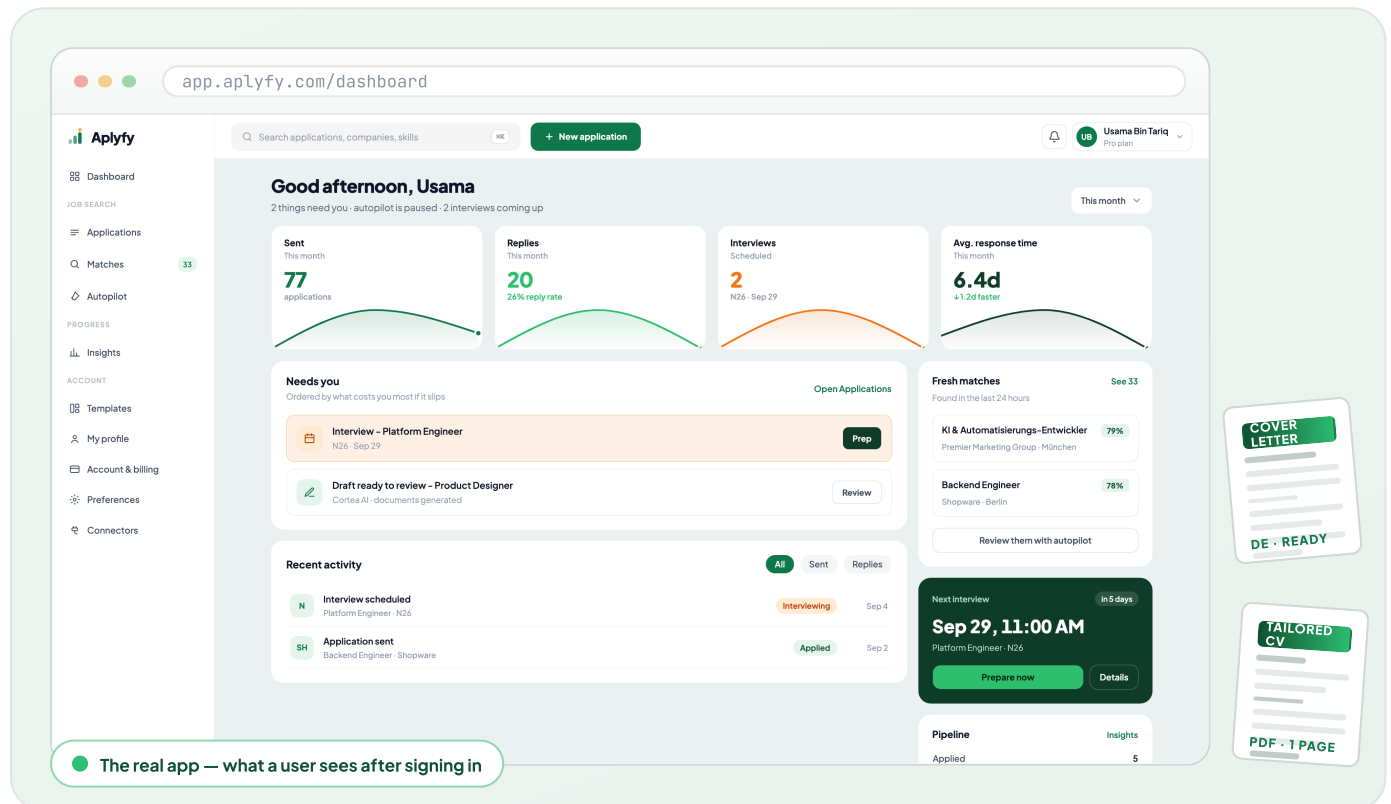
Full-stack product

AI pipeline

AWS serverless platform

Design system

EN / DE



196k

lines shipped across 7 projects

< 60s

job posting → finished PDFs

9

background lanes, each with a safety net

~\$11

per month to run the database tier

React 19

Vite

Tailwind v4

shadcn/ui

NestJS

Prisma

PostgreSQL

AWS Lambda

SQS

S3 Vectors

Bedrock Nova

Gemini

Playwright

Puppeteer

Next.js

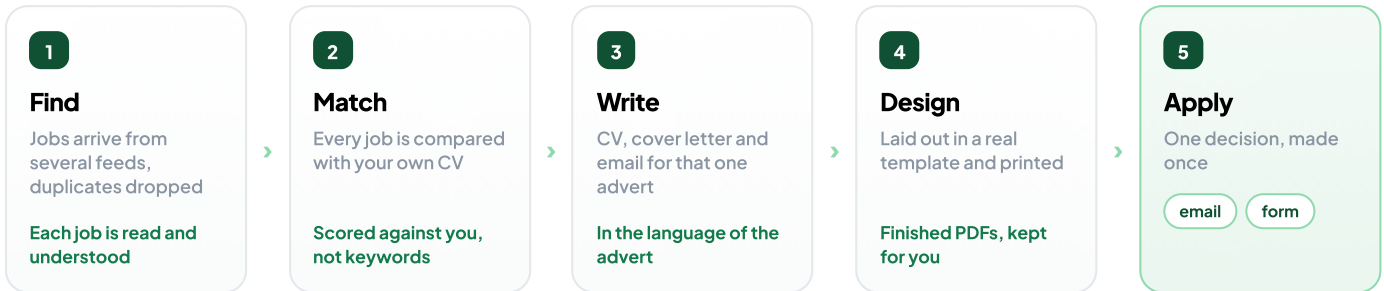
CircleCI



THE USER'S MINUTE

Five steps, none of them yours

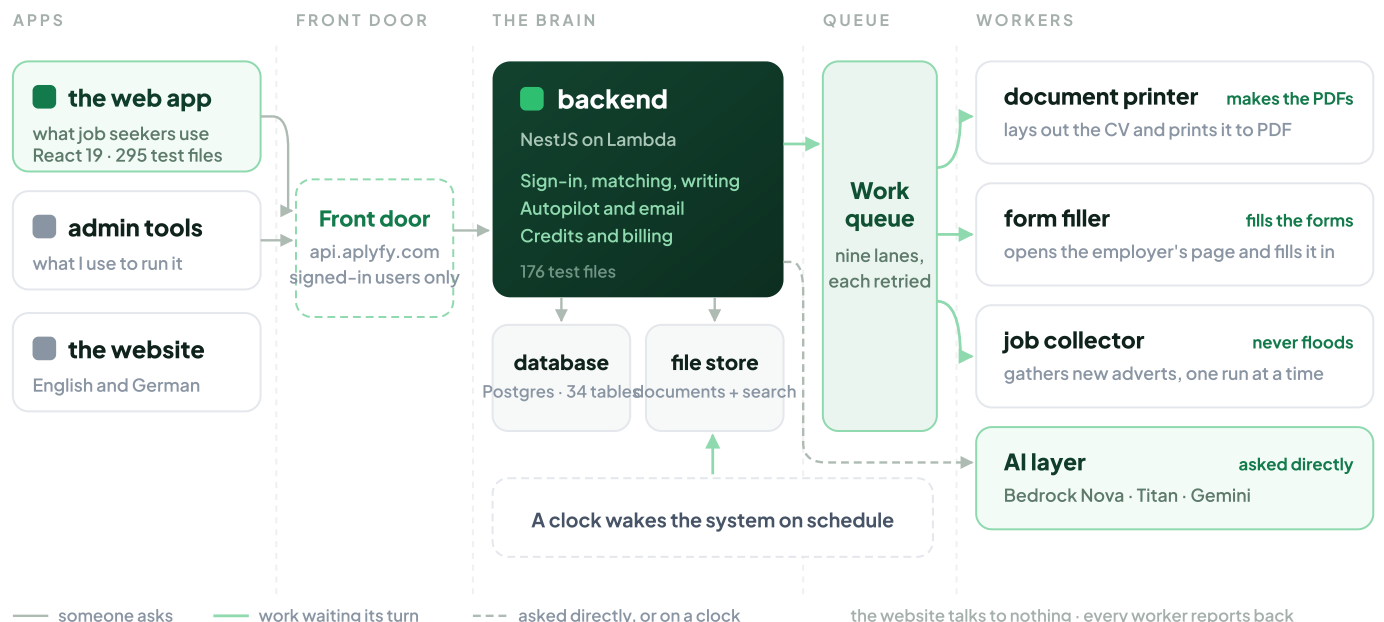
Each step retries by itself; anything stuck is set aside for me



UNDER IT

One hub, many arms

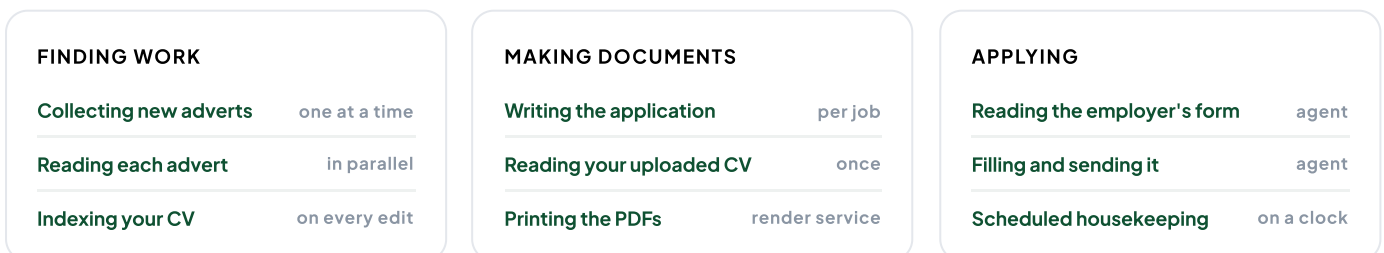
No servers to keep alive — it costs nothing while nobody is using it



NOTHING WAITS ON A PERSON

Nine lanes of background work

A failed job is retried twice, then parked where I can see it





THE LAST MILE

An agent fills the form

It opens the real page, reads every question, and answers what it can

The screenshot shows the Aplyfy Autopilot interface. The top navigation bar includes the Aplyfy logo, a search bar, a 'New application' button, and a user profile dropdown for Usama Bin Tariq. The left sidebar contains navigation links: Dashboard, Applications, Matches, Autopilot (highlighted), Insights, Account, and Connectors. The main content area is titled 'Autopilot' and shows '12 jobs waiting for you'. A progress bar indicates 'JOBS 1 OF 12'. The job details for 'Senior Backend Engineer' are displayed, including a 'Form captured' button and a 'View details' link. The job description is visible, and the form fields for 'Full name', 'Email', 'CV upload', and 'Work authorization' are shown. A sidebar on the right provides a detailed view of the application process, including steps like 'Documents', 'Field mapping', 'Submission', and 'Verification'.

- ✓ **Read first, type second.** The whole form is understood before a single box is filled.
- ✓ **Practice run by default.** It fills the form and takes a picture without sending.
- ✓ **Email or form, decided once.** If it says email, an address exists — the database refuses anything else.
- ✓ **Every question is sorted.** Known answer, fact about you, or a question only you can answer.
- ✓ **Never without the CV.** Two separate checks refuse to send an application without it.
- ✓ **Every attempt is on record.** What was sent, when, and a screenshot of the page.

Every application moves through twelve states — none of them a dead end

The horizontal bar chart displays the twelve states of an application. The states are: Writing, Documents ready, Needs your answer, Queued, Filling in, Checking, Sent, and Failed, retried. The 'Needs your answer' state is highlighted in orange, indicating it is the current state of the application.

BUILT TO BE CHANGED

Safe to touch, six months later

Nothing reaches users until the tests agree

Tests on the app	295
Screens in a catalogue	218
Tests on the API	176
Database changes	35
Tests on the form filler	16

DECISIONS I'D DEFEND

- Wording kept in files, not in code
- Shared types one vocabulary, both ends
- API clients generated, never hand-written
- Screens one request each, not twenty
- Releases swap the image, keep the settings
- Claims in a CV checked, not trusted



SELECTED WORK · 02

Visorun

A CV anonymisation platform for recruiters — designed, built and run by me.

A recruiter drops in a candidate CV. Visorun reads it, strips every identifying detail and returns a branded PDF the client can see — with plans, quotas and Stripe billing around it.

My role Everything: product and UI, NestJS API, Firestore model, Stripe, SendGrid, Cloud Functions, deploys.

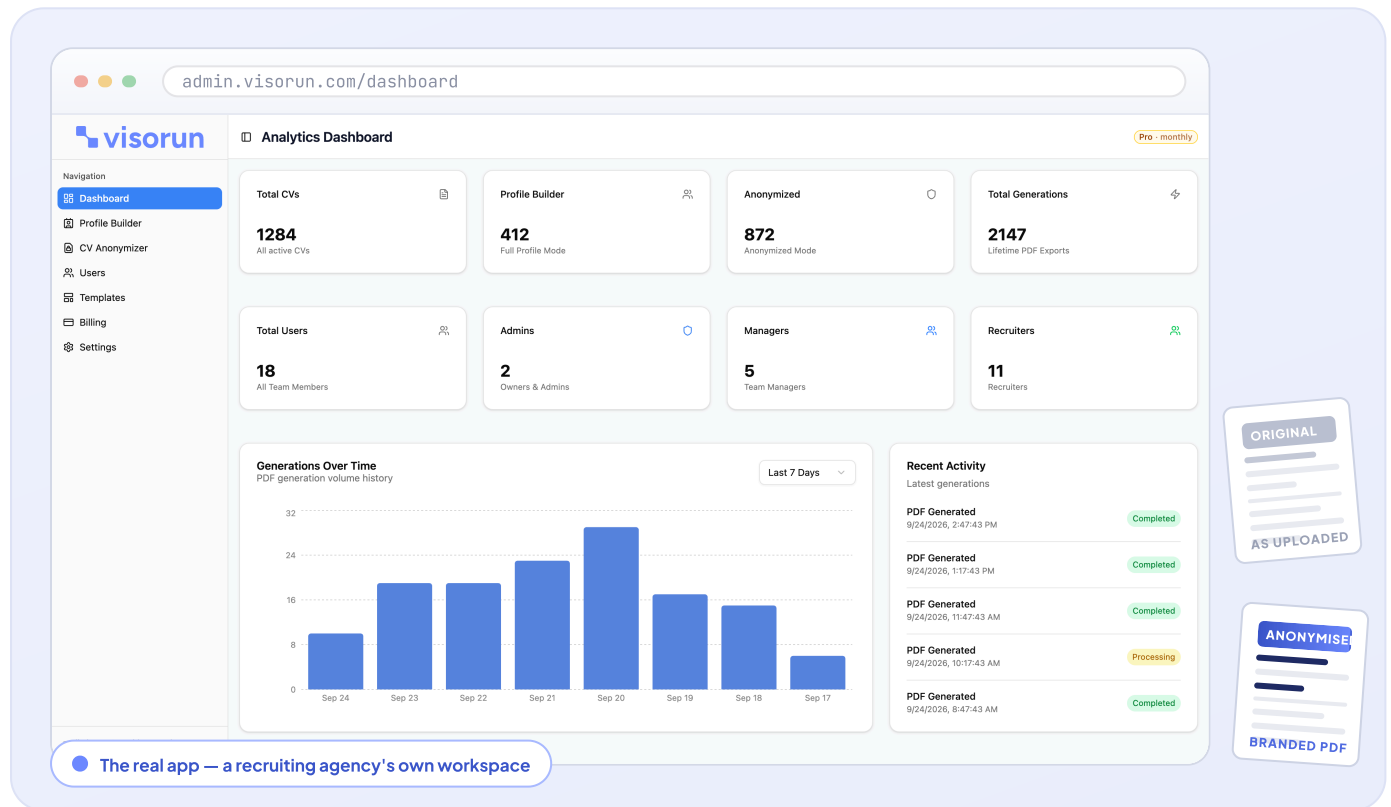
Multi-tenant SaaS

Stripe billing

AI extraction

Plan gating

EN / DE



64k

lines across 8 workspaces

25

milestones, each with its own log

7

branded CV templates, EN + DE

3+4

plans and top-up packs, all in Stripe

React 19

Vite 7

Tailwind 4

Radix

NestJS

Firestore

Functions v2

Cloud Run

Stripe

SendGrid

Gemini

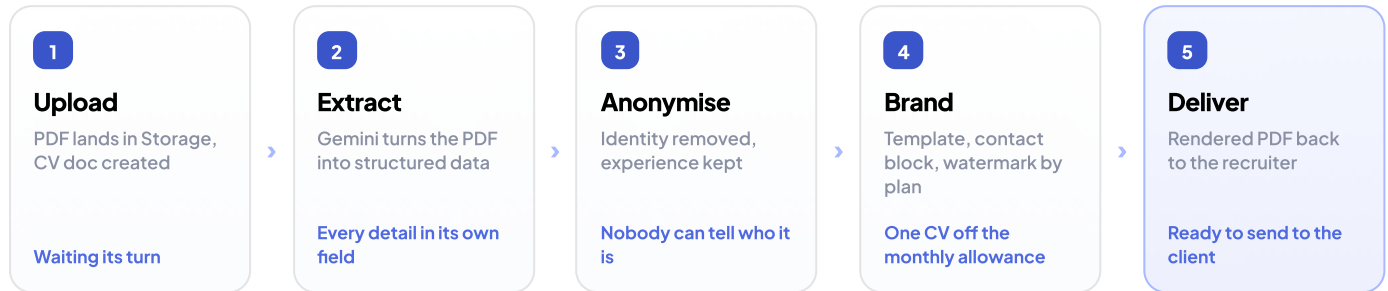
Puppeteer

Yarn 4

ONE UPLOAD

Five steps, no babysitting

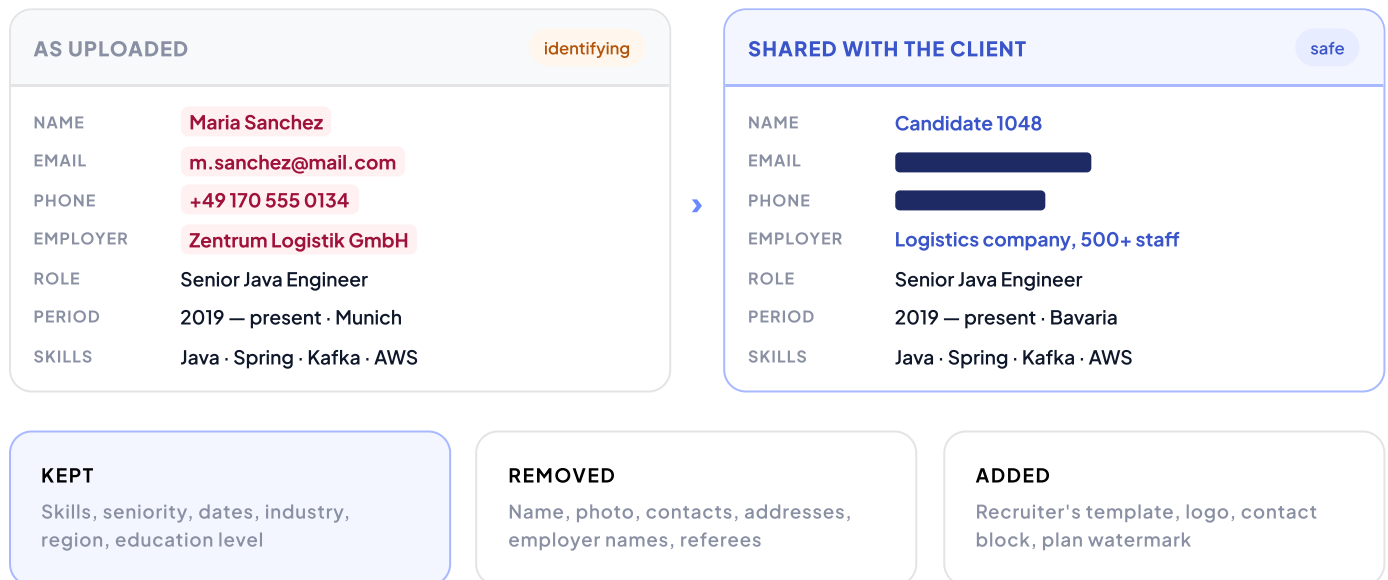
Each step is a trigger · quota is consumed once, atomically



THE CORE OF THE PRODUCT

Identity out, evidence in

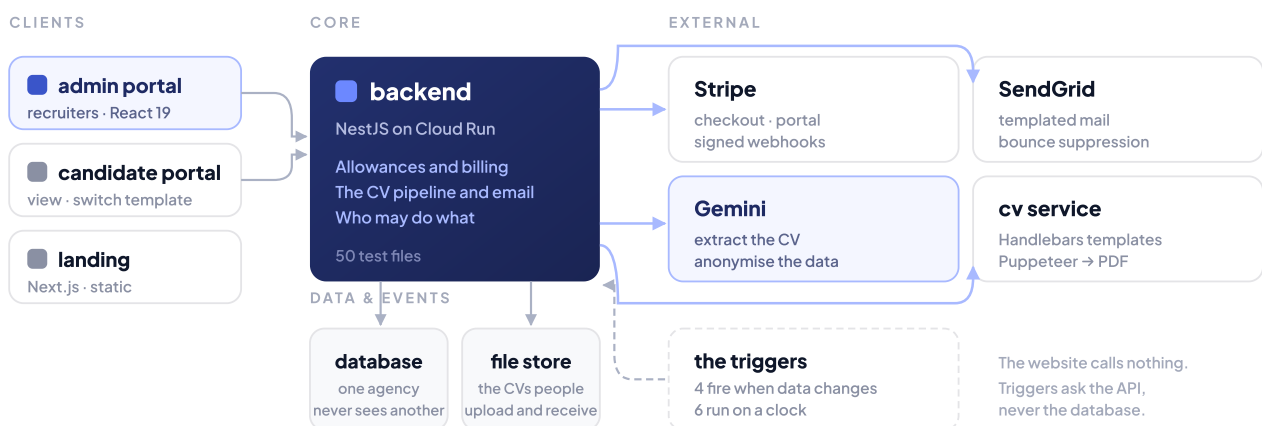
Skills, dates and seniority survive · names, contacts, employers don't



UNDER IT

One API, everything else is a plug

Functions stay thin · all business logic lives in the API





EVERY FEATURE HAS A PRICE TAG

Three plans, one source of truth

Limits live in one shared constant · the API refuses, the UI explains

admin.visorun.com/billing

visorun

Navigation

- Dashboard
- Profile Builder
- CV Anonymizer
- Users
- Templates
- Billing**
- Settings

Billing

Usage this cycle

137 / 200 CVs this cycle · 63 left · +25 top-up credits

Subscription overview

Plan	Pro (pro)	Billing cycle	monthly
Base subscription	\$149 / month	Next payment	October 13, 2026
Paid Recruiter seats	2 active	Cost per seat	\$30 / month
Seat add-ons total	\$60 / month	Total per month (incl. seats)	\$209 / month

Paid Recruiter seats are billed on the same cadence as your plan (monthly) and prorated when added mid cycle. Removing a Recruiter keeps the paid seat — remove the empty seat under "Paid seats" to stop being charged for it.

Paid seats

You have 2 paid seats · 2 in use · 0 empty. New seats are added when you invite a user beyond your plan's limit. Removing a Recruiter keeps their paid seat — remove the empty seat here to stop being charged for it.

Free · 5 CVs a month

Starter · 50 a month, extra seat

Pro · 200 a month, 3 people

Top-ups in packs of 10, 25 and 100

What Stripe tells the app, and what the app does about it

Paid

Payment failed

Reminder in 3 days

Plan changed

Allowance reset

Cancelled

Access ends

RUNS WITHOUT ME

Six jobs that run on a clock

Reminders, warnings and clean-up happen whether anyone logs in or not

Send waiting emails every minute

Warn before the limit 09:00

Follow up a cancellation 09:15

Credits about to expire 09:30

Nudge quiet accounts Mondays

Expire old invitations 03:00

- ✓ **An allowance is never spent twice.** The month goes first, then the oldest credits — in a single step.
- ✓ **Stripe may repeat itself safely.** Every message is checked and remembered, so nothing is applied twice.
- ✓ **Dead addresses stop costing.** Anything that bounces or is marked spam is never mailed again.
- ✓ **Seats follow the plan.** Move to a smaller plan and the next invite is caught, not a support ticket.
- ✓ **Locked means "upgrade", not "error".** Every blocked action explains itself the same way.

DECISIONS I'D DEFEND

What each plan allows	written down once
Money and limits	only the API may change them
Triggers	tiny; the thinking lives in one place
Every email	queued, retried, recorded
Payments	tested against a fake Stripe
Each milestone	left a written record

MAIL THAT HAS TO ARRIVE

Queued, retried, suppressed

Five attempts with backoff · every send is logged against the org

Queued

Sent

Delivered

Bounced

Marked as spam

Never mailed again



SELECTED WORK · 03

Stuzanne

A **workshop system for a jewellery maker** — built, packaged and shipped by me.

Every commissioned piece is a job: who is making it, what it must become, when it is due.
Gold goes out to a craftsman and gold must come back — including the filings. The workshop runs its day from this, on the desktop.

My role Everything: the domain model, .NET API, MySQL schema, React app, desktop packaging and releases.

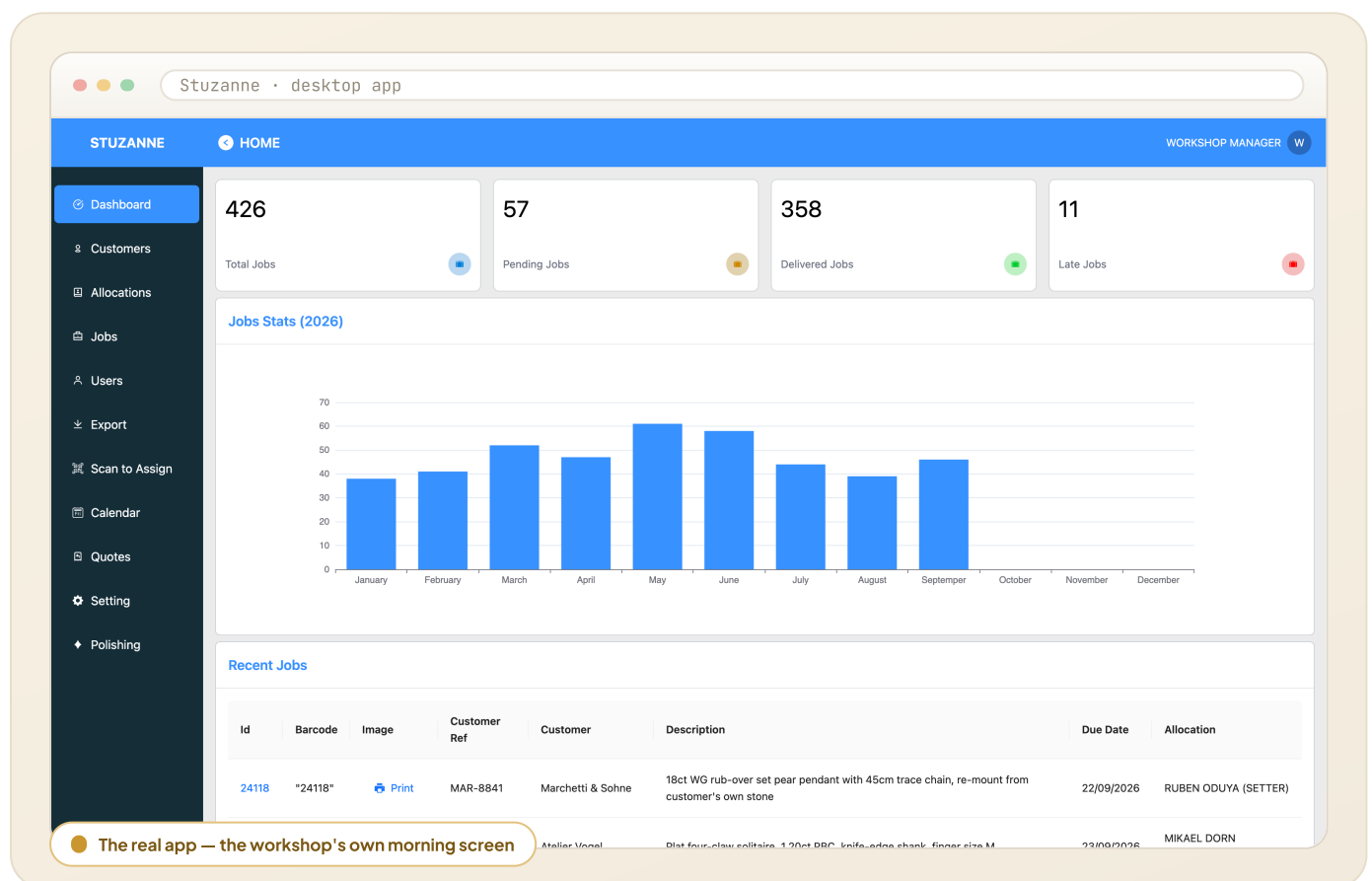
Manufacturing ERP

Domain modelling

Desktop app

Barcode workflow

Real client, in production



The real app — the workshop's own morning screen

17

screens, from quoting to polishing

36

API endpoints behind them

8

alloys tracked separately per job

1 click

from merge to a signed Windows installer

.NET 6

EF Core

MySQL

React 18

TypeScript

Ant Design

Electron

Docker

JWT

S3

GitHub Actions



THE HEART OF IT

Gold out, gold back — to the gram

Filings count. Off-cuts count. What is missing is the number that matters.

ISSUED TO THE BENCH		what left the safe
Yellow gold	9.40 g	
White gold	2.10 g	
Rose gold · platinum	0.00 g	
Silver, half alloys	0.00 g	
Customer's own piece	3.45 g	
Bought-in bits	2.00 g	
Total issued		16.95 g

RETURNED BY THE BENCH		what came back
Finished piece	13.10 g	
Lemel — filings and dust	2.35 g	
Off-cuts, first return	0.95 g	
Off-cuts, second return	0.41 g	
Stones are counted, not weighed		4 set
Hallmarking sent		yes
Total returned		16.81 g

0.14 g short

That gap is the whole product. Over hundreds of jobs it is real money, and before this system nobody could point to the job it went missing on.

THE SAME JOB, IN THE APP

Who held it, and what came back

Every hand-over and every weighing is kept, so a loss has an owner

Stuzanne · job detail

Customers

Allocations

Jobs

Users

Export

Scan to Assign

Calendar

Quotes

Setting

Polishing

History

Id	Allocation	Allocation Date	Returning Date	Action
2411900	NOUR HADDAD (CAD_DESIGNER)	06/09/2026 08:30	08/09/2026	
2411901	TOMAS BERGER (MOUNTER)	09/09/2026 08:30	11/09/2026	
2411902	RUBEN ODUYA (SETTER)	12/09/2026 08:30	14/09/2026	
2411903	MIKAEL DORN (POLISHER)	15/09/2026 08:30	No Date	

< 1 >

Metal Calculations

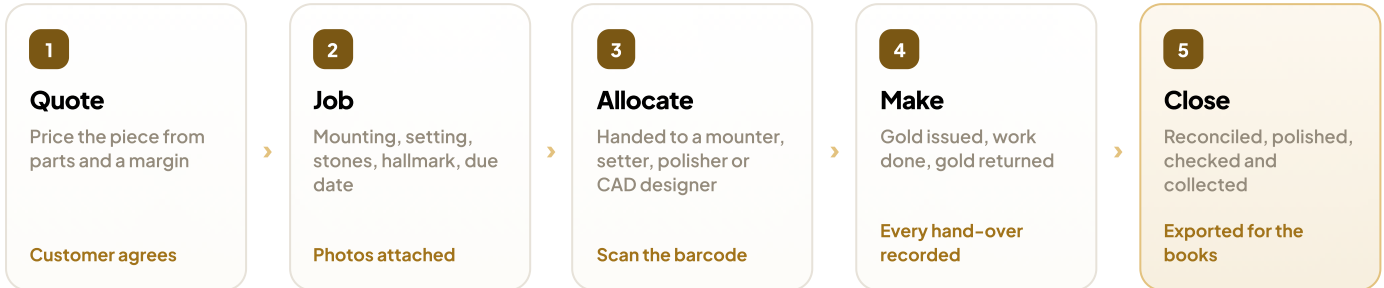
Metal Given	16.95	Metal Returned	16.81	Difference	0.14	Percentage	0.83
-------------	-------	----------------	-------	------------	------	------------	------

Metal History

ONE COMMISSION

From quote to collected

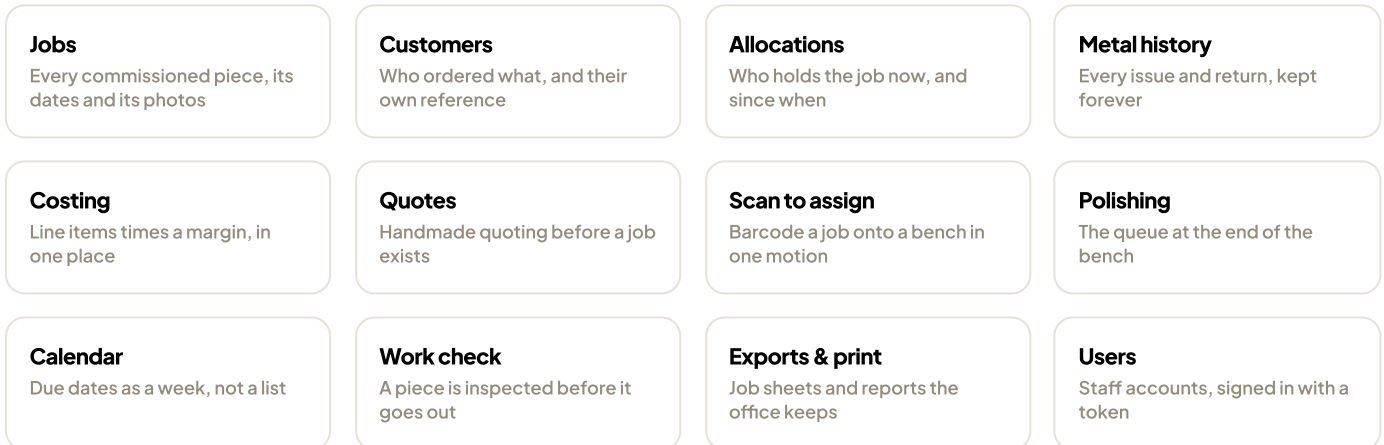
Each step is a screen the workshop actually uses



ONE WORKSHOP, TWELVE JOBS TO DO

What the app covers

Each is a screen with an API behind it, not a menu stub



DELIVERY

A web app that arrives as a desktop app

Merge to main, and the workshop gets an installer



- ✓ **The sums live in the model.** Totals are computed from the parts, so no screen can disagree with another.
- ✓ **Nothing is overwritten.** Metal movements are appended, so last month can still be explained.
- ✓ **The bench uses a scanner.** Assigning work is a barcode, not a form.
- ✓ **One app, two shapes.** The same build runs in a browser and as an installed Windows program.

DECISIONS I'D DEFEND

The domain	modelled on the bench, not on a CRUD table
Eight alloys	separate columns, never one "gold"
Losses	a first-class number, not a report
Auth	tokens, every endpoint closed by default
Releases	automated, because the client is not technical



SELECTED WORK · 04

LR Car Services

The **driver's app** for a dispatcher-run car service — built and shipped by me.

A dispatcher assigns a trip; the driver's phone takes it from there. Multi-stop routes, waiting time counted separately from driving time, and a location feed that keeps reporting even when the app has been killed.

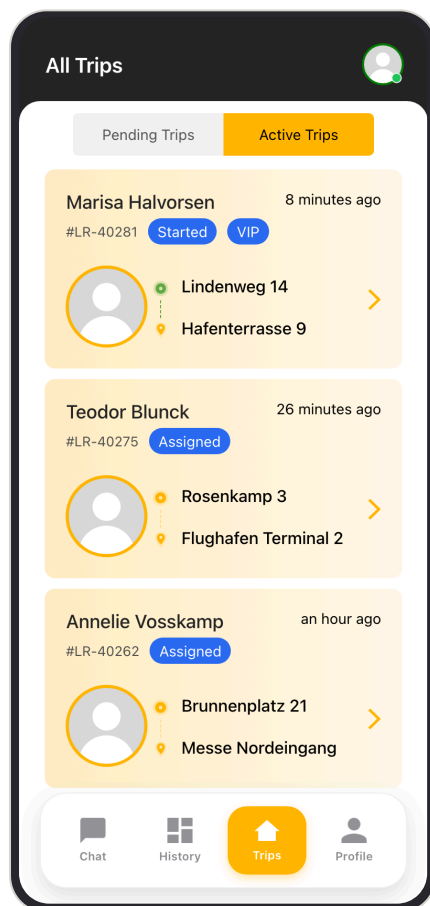
My role Everything: the React Native app, the cloud functions, the data model and the releases.

Mobile, Android + iOS

Real-time dispatch

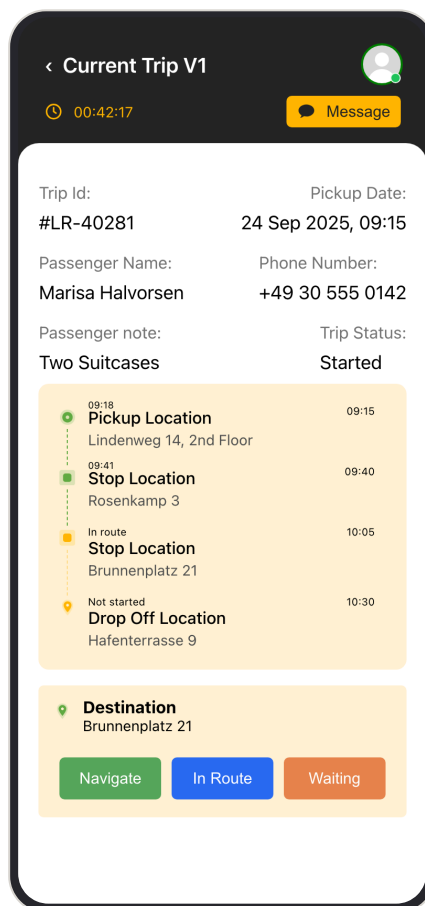
Background GPS

Push & chat



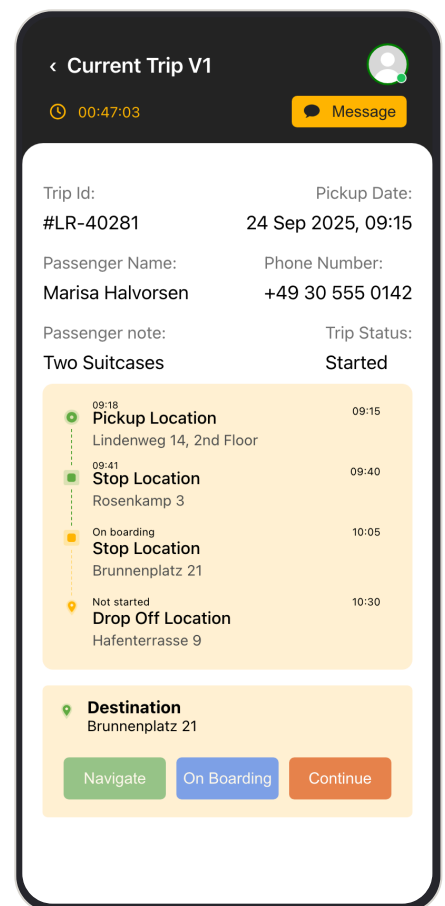
The day's work

Assigned and active trips, four tabs



A trip in progress

Pickup, stops and drop-off, each with its own state



Waiting at a stop

The clock keeps running, on a separate meter

Screens drawn from the app's own source — every label, state and button is one that ships

20

screens in the driver app

5

separate cloud services behind it

60s

without a ping and a driver shows offline

v8

shipped to real drivers on both stores

React Native

React 19

TypeScript

Firebase Auth

Firestore

Cloud Functions

FCM push

Background GPS

Google Cloud



THE TRIP ITSELF

Five states, one direction

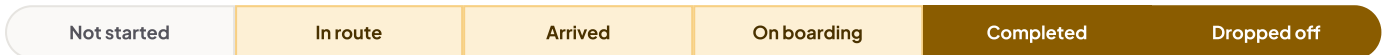
The dispatcher opens it, the driver moves it forward, nobody skips a step



AND INSIDE IT

Every stop has its own six

Pickup, stop, drop-off — each tracked, each timed



< Current Trip V1

Message

Trip Id: #LR-40281
Pickup Date: 24 Sep 2025, 09:15
Passenger Name: Marisa Halvorsen
Phone Number: +49 30 555 0142
Passenger note: Two Suitcases
Trip Status: Completed

09:18
Pickup Location
Lindenweg 14, 2nd Floor

09:41
Stop Location
Rosenkamp 3

10:12
Stop Location
Brunnenplatz 21

10:37
Drop Off Location
Hafenterrasse 9

Waiting Time: 00:14:22
Total Time: 01:22:05

Back To Trips

WHY WAITING IS COUNTED ON ITS OWN

A chauffeur job is half driving and half waiting — outside a meeting, at an airport, at a restaurant. The driver taps **Waiting**, and taps **Continue** when they move again. Each pause is stored, then added up.



Two figures, not one. The office can bill the wait, and a driver who sat for forty minutes can prove it.

Partial drop-offs

Passengers can leave at different stops — the trip carries on without them

Old trips keep the old app

Two generations of the trip screen live side by side, chosen per trip

A line to the office

Driver and dispatcher chat inside the trip; both sides get a notification

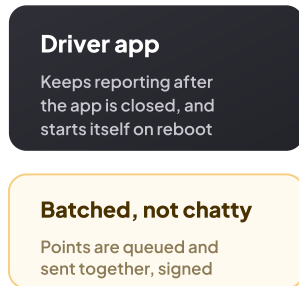


THE HARD PART OF A DRIVER APP

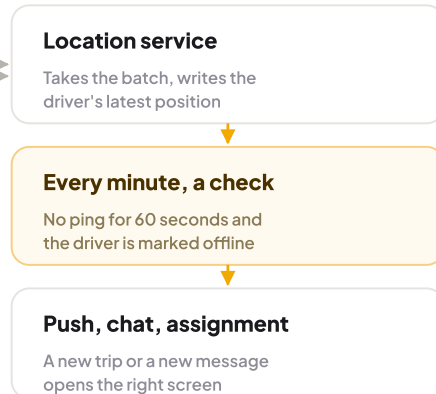
A phone in a pocket, all day

Killed apps, dead batteries, tunnels — none of it can lose the trip

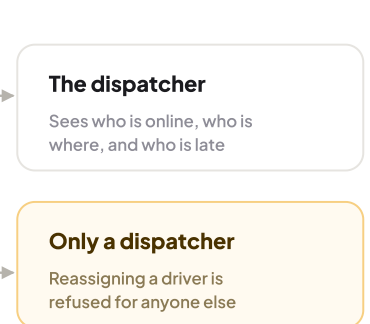
THE PHONE



THE SERVICES



WHO SEES IT



- ✓ **Reassignment is a privilege.** The server checks the caller is a dispatcher and refuses everyone else — the app being polite is not enough.
- ✓ **Drivers are approved, not just registered.** Pending, approved or blocked, decided by the office.
- ✓ **Addresses are not hard-coded.** The app reads where its services live at start-up, so an endpoint can move without a store release.
- ✓ **Updates can be made compulsory.** A driver on a broken old build is a lost trip, so Android can force the update.
- ✓ **Trips already running never change shape.** A new trip screen shipped beside the old one instead of replacing it.

DECISIONS I'D DEFEND

Waiting time	its own number, from day one
A trip	a chain of stops, not A to B
Location	batched and signed, not streamed
Offline	decided by the server's clock
Old versions	kept working, not forced out

Trips the work itself

Drivers who may drive

Driver status online, permissions

Locations where they are

Chats office and road

Settings what the app reads